

Apuntes Día 1

CURSO DE DESARROLLO CON IA

El Nuevo Programador

Índice

¿Cuál es el impacto real de la IA?	3
Inicios de 2026	3
2026 (actualidad)	4
Mercado laboral	4
Las claves	5
La realidad	6
Los temores	6
El Nuevo Programador	7
01 - Fundamentos de la Inteligencia Artificial	7
¿Qué es un LLM y cómo funciona?	7
Fundamentos clave de los LLM	8
Tipos de modelos	9
Utilización	9
Herramientas	9
Prompt engineering (clásico)	10
Prompt engineering (2026)	10
Vibe coding vs. Ingeniería de software	11
02 - Harness engineering (arneses y guardarraíles)	12
¿Qué es un Agente?	12
OpenCode	13
¿Qué es?	13
Características	13
Instalación	13
Comandos y atajos más utilizados	14
Prompt utilizado para la creación inicial de la aplicación de ejemplo	14
Context engineering	15
AGENTS.md	16
Reglas, Memoria y Documentación	17
Modo Plan (Agente)	18
Súmate a nuestro directo del día 2	20

¿Cuál es el impacto real de la IA?



Educación

Nunca fue tan sencillo y rápido empezar a desarrollar software.



Desarrollo

La IA genera el código. Tú garantizas la arquitectura. El nuevo desarrollador no teclea más, decide mejor.



Empleo

El nuevo estándar del mercado: Fundamentos de Ingeniería + Inteligencia Artificial.

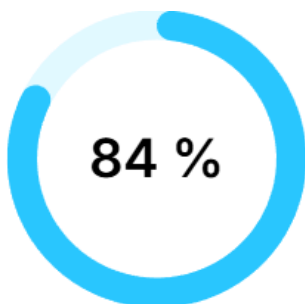


Futuro

No te reemplaza la IA. Te reemplaza un desarrollador que sabe dirigirla con criterio profesional.

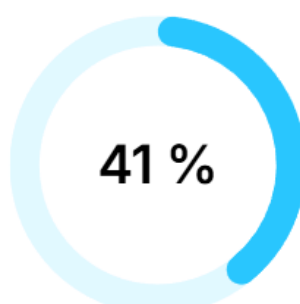
Inicios de 2026

Nos encontramos ante el cambio estratégico más profundo de la última década en el sector tecnológico: el tránsito definitivo del acto de "escribir código" al de "dirigir inteligencia artificial". No estamos ante la obsolescencia del programador, sino ante su evolución hacia un rol de supervisor crítico, donde la IA genera el volumen y el humano garantiza la veracidad y la seguridad.



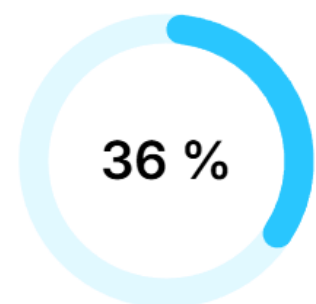
Developers

Utilizan IA para programar en su día a día



Código

"Nuevo" generado está escrito por IA

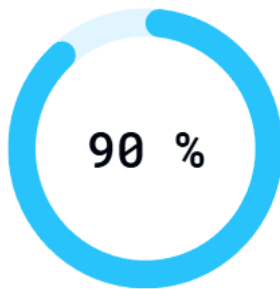


Confianza

En los resultados producidos por IA **sin supervisión**

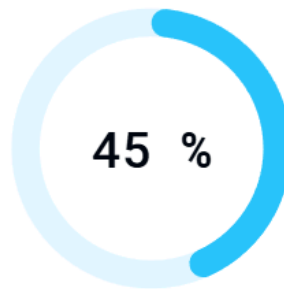
Datos extraídos de diferentes informes de Google, Microsoft, GitHub, JetBrains y Stack Overflow

2026 (actualidad)



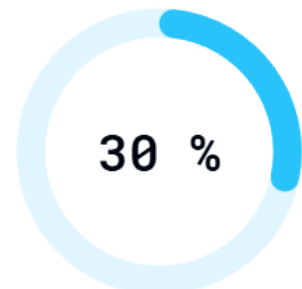
Developers

Utilizan IA para programar
en su día a día



Código

“Nuevo” generado está
escrito por IA



Confianza

En los resultados
producidos por IA **sin
supervisión**

Datos extraídos de diferentes informes de Google, Microsoft, GitHub, JetBrains y Stack Overflow

Mercado laboral

Software Development Job Postings Have Been Experiencing a Rebound

United States, January 2025 - June 26, 2026, seasonally adjusted | Index, 24 Feb 2025 = 100



Source: Indeed.

indeed

<https://hiringlab.indeed.com/2026/07/08/ai-and-job-postings-from-destruction-to-creation>

**El sector del desarrollo de software ha
cambiado para siempre.**



**Lenguaje de
programación
+ Framework**



**Fundamentos
+ Inteligencia
Artificial**

Las claves

- Crear software no es programar.
- Los fundamentos importan más que nunca.
- Tú diriges, la IA ejecuta.
- Utilizar IA no es opcional.

La realidad



El mercado ya no paga por teclear, sino por orquestar.

Los temores



**Por donde
empezar**



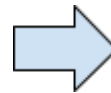
**Es muy
complicado**



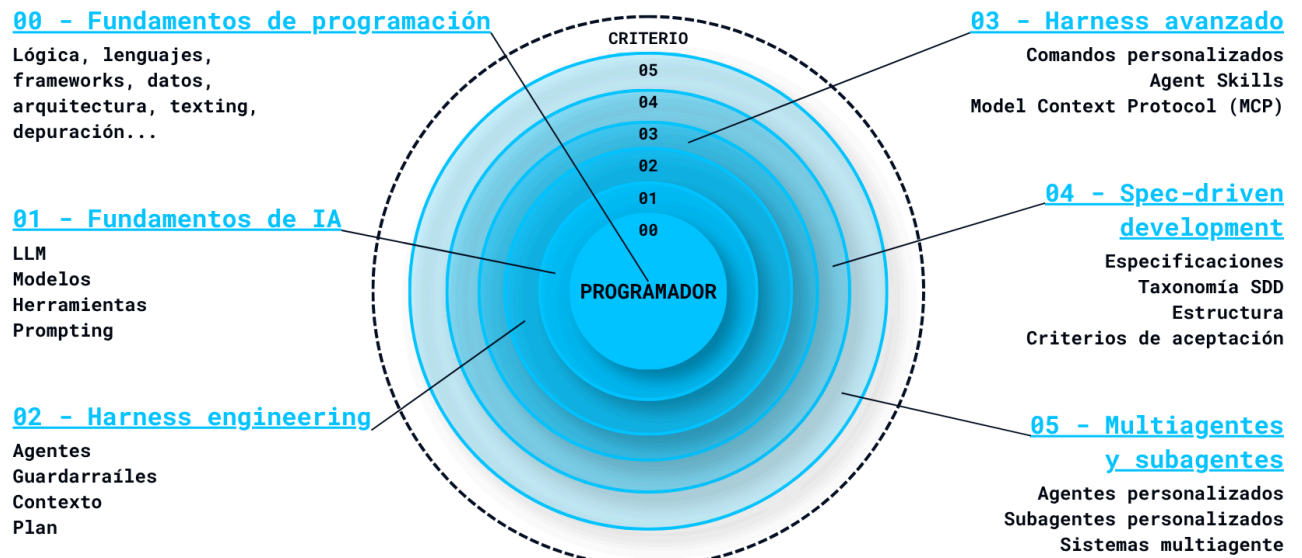
**Es solo
para seniors**



**Esto es
el futuro**



El Nuevo Programador



01 - Fundamentos de la Inteligencia Artificial

¿Qué es un LLM y cómo funciona?

Large Language Model (LLM): Es un modelo de IA entrenado con grandes cantidades de texto para aprender patrones del lenguaje y generar respuestas mediante probabilidad (inferencia).

Funcionamiento:

1. Recibe el mensaje
2. Lo procesa en fragmentos llamados tokens
3. Calcula qué token podría venir a continuación, elige uno y repite el proceso hasta completar la respuesta.

Predice el siguiente token. No "piensa", calcula probabilidades.

Fundamentos clave de los LLM

Número de parámetros: Son las "conexiones neuronales" del modelo que almacenan lo que ha aprendido. Cuantos más parámetros tiene un modelo, mayor es su capacidad para representar conocimiento y patrones complejos.

Tokens: Unidades en las que se procesa la información. Los LLM no leen palabras, leen y calculan tokens, y es la medida que se usa para cobrar su uso.

Temperatura: Ajuste al seleccionar tokens. Controla la aleatoriedad del texto generado por los LLM durante la inferencia.

Prompt: Instrucciones que proporcionas al modelo.

Ventana de Contexto: Es la "memoria a corto plazo" del modelo en una conversación. Define la cantidad máxima de texto (tokens) que el modelo puede leer, procesar y recordar a la vez sin "olvidar" el principio del documento o la charla. Si un proyecto excede este límite, la IA empezará a "olvidar" requisitos previos o a alucinar, lo que hace indispensable una gestión estratégica de la información enviada al modelo.

Multimodalidad: Capacidad de procesamiento. Un modelo multimodal puede "ver" imágenes, "escuchar" audios, analizar vídeos.

Razonamiento vs. Rapidez: Diferencia entre modelos convencionales y los nuevos modelos de razonamiento que pausan, planifican y verifican internamente sus pasos lógicos antes de darte la respuesta final.

Alucinaciones: Es el mayor defecto de los LLM. Ocurre cuando el modelo no sabe la respuesta y se inventa información.

Latencia: Es el tiempo de espera. Se mide desde que pulsas "Enviar" hasta que el modelo responde el primer token.

Tipos de modelos

- **Pesos abiertos/cerrados:** Si puedes acceder a los parámetros del modelo. Su uso depende de la licencia.
- **Ejecución nube/local:** Si el modelo se ejecuta en tu equipo o en servidores remotos.

Un modelo de pesos abiertos puede ejecutarse tanto en local como en la nube.

Los de pesos cerrados suelen ofrecerse mediante servicios en la nube.

Pesos abiertos no significa necesariamente código abierto. Puede faltar el código de entrenamiento, los datos o permisos de uso.

<https://artificialanalysis.ai>

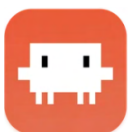
<https://llm-stats.com/>

Utilización

Potencia ↔ Velocidad ↔ Coste: Elegir según tarea.

- Modelo potente para pensar y arquitectura.
- Modelo medio para el día a día.
- Modelo rápido y barato para tareas mecánicas.

Herramientas



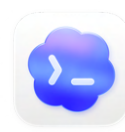
[Claude Code](#)



[OpenCode](#)



[Warp](#)



[Codex](#)



[ChatGPT](#)



[GitHub Copilot](#)



[Cursor](#)

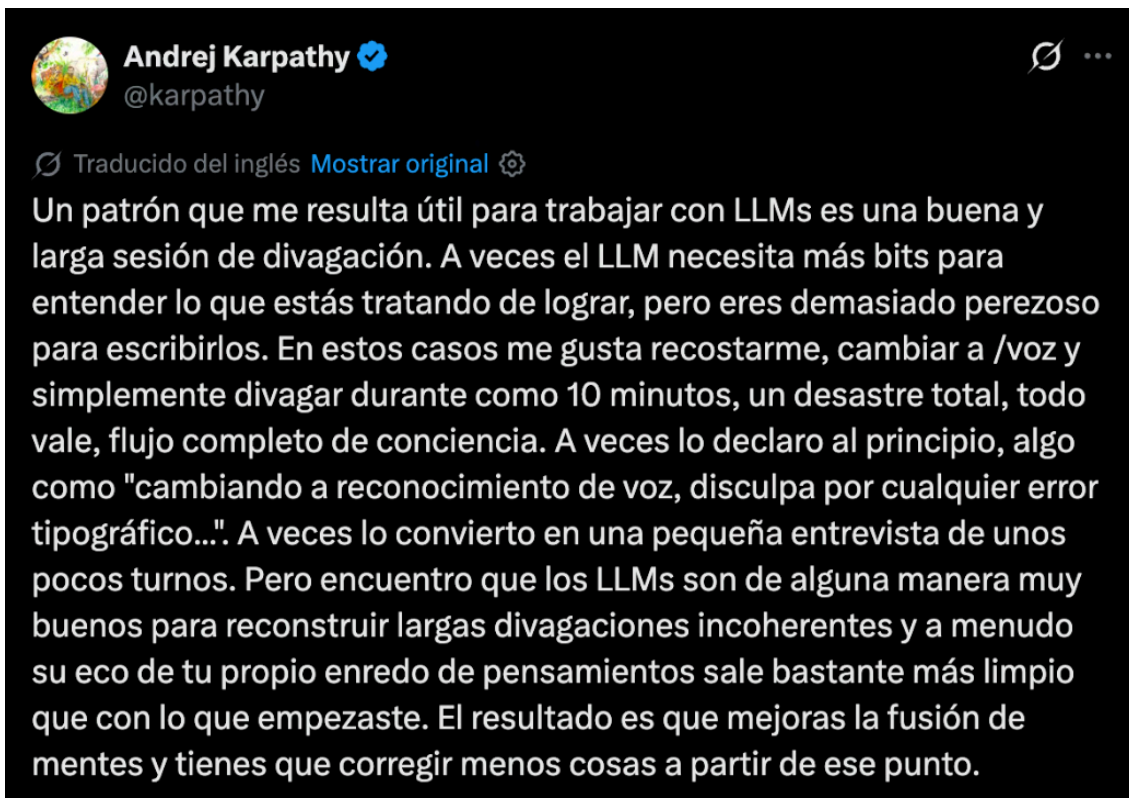


[Antigravity](#)

Prompt engineering (clásico)

1. **Rol (¿Quién soy?):** Define el nivel de experiencia y la especialidad que quieres que asuma la IA.
2. **Contexto (¿Dónde estamos?):** Explica de qué trata el proyecto, tecnologías y cuál es el problema general.
3. **Tarea exacta (¿Qué necesitas?):** Sé específico. En lugar de pedir un "sistema" algo concreto.
4. **Restricciones o Reglas (¿Qué límites hay?):** Indica convenciones y estándares.
5. **Formato de salida (¿Cómo lo quieres?):** Cómo quieres recibir la información.

Prompt engineering (2026)



Vibe coding vs. Ingeniería de software

- **Vibe coding:** programar guiándose por intuición y ayuda de IA para generar código rápido. Describir y aceptar sin revisar. Rápido, divertido y peligroso.
- **Ingeniería de software:** diseñar y construir software de forma sistemática con arquitectura, control, pruebas y mantenimiento a largo plazo.



02 - Harness engineering (arneses y guardarraíles)

Para un profesional de la ingeniería de software, seguir utilizando un chat convencional para programar es una ineficiencia estratégica. El salto cualitativo hacia los flujos agénticos permite que el desarrollador opere en un nivel de abstracción superior, validando intenciones en lugar de corregir líneas de código aisladas.

¿Qué es un Agente?

Es un sistema que utiliza un modelo para realizar tareas orientadas a un objetivo, encadenando decisiones y acciones.

Puede planificar, consultar información, usar herramientas, modificar archivos, ejecutar código y comprobar resultados, según los permisos disponibles y bajo tu supervisión.

El bucle del Agente



Diferencias entre Chat y Agente

Chat	Agente
• Pregunta → Respuesta • Sin acceso al sistema • Un solo turno • Requiere copy-paste	• Bucle autónomo • Lee/escribe archivos • Ejecuta y verifica • Itera

OpenCode

¿Qué es?

Es un agente de código de IA de código abierto. Está disponible como interfaz basada en terminal, aplicación de escritorio o extensión IDE. Es la alternativa a Claude Code.



Características

- Open-source
- Agente de su categoría más utilizado
- Agnóstico al modelo
- Tiene una capa de uso gratis

Web Oficial: <https://opencode.ai>

Repositorio Oficial: <https://github.com/anomalyco/opencode>

OpenCode Zen: <https://opencode.ai/es/zen>

OpenCode Go: <https://opencode.ai/es/go>

Instalación

- **Descarga e instalación:** <https://opencode.ai/es/download>
- **Documentación Oficial:** <https://opencode.ai/docs>
- **Iniciar OpenCode:** desde la terminal escribir el comando **opencode**

Comandos y atajos más utilizados

- **opencode web** → Abre una interfaz web en local para usar en lugar de la terminal
- **/help** → Guía de referencia rápida y catálogo de funciones
- **/connect** → Selecciona los proveedores de IA a utilizar
- **/models** → Selecciona los modelos a utilizar en cada proveedor de IA
- **/init** → Analiza el proyecto y genera automáticamente el archivo agents.md
- **/agents** o **TAB** → Cambia entre modo Build y Plan
- **/status** → Ver el estado de la sesión
- **/new** → Abre un nuevo chat
- **/session** → Lista las sesiones de chat
- **/compact** → Compacta la sesión
- **/undo** → Deshace un mensaje
- **/redo** → Rehace un mensaje
- **/diff** → Muestra los cambios en un archivo de código luego de la última ejecución
- **exit** → Cierra la sesión actual
- **CTRL + P** → Muestra en detalle todos los comandos de OpenCode
- **CTRL + T** → Cambia el nivel de esfuerzo del modelo de IA elegido
- **@** → Referencia a ficheros, carpetas, skills, etc.
- **!** → Modo Shell (ejecuta comandos de terminal sin salir de OpenCode)

Prompt utilizado para la creación inicial de la aplicación de ejemplo

None

Rol

Actúa como desarrollador frontend senior que escribe código simple, claro y fácil de entender para alguien que está empezando a programar.

Contexto

Quiero crear desde cero "Diario de Estudio", una web para registrar mis sesiones de estudio y motivarme viendo mi racha de días seguidos estudiando. Esta es la primera versión y tiene que ser muy simple. La web se construirá poco a poco, así que ahora solo necesito una base limpia que funcione a la primera.

Tarea

Crea la web con estas funcionalidades:

1. Un formulario para registrar una sesión con:

- Fecha (por defecto hoy, pero editable para poder apuntar días anteriores)
- Tema (texto, obligatorio)
- Minutos (número mayor que 0, obligatorio)

```
2. La racha actual en grande, con un 🔥.
3. La lista de sesiones, de la más reciente a la más antigua.
4. Los datos guardados en localStorage para que no se pierdan al recargar.

## Restricciones y reglas
Racha:
- Un día cuenta si tiene al menos una sesión.
- La racha son los días consecutivos con sesión que terminan hoy.
- Si hoy todavía no he estudiado pero ayer sí, la racha sigue viva: no se rompe hasta que termina el día.
- Usa siempre la fecha local del usuario, nunca UTC.

Técnicas:
- HTML, CSS y JavaScript, sin frameworks, sin librerías y sin compilar nada.
- Solo tres archivos: index.html, styles.css y app.js.
- Tiene que funcionar abriendo index.html con doble clic, sin servidor ni instalación.
- No añadas nada que no aparezca en este mensaje.
- Diseño limpio y moderno, que se vea bien en el móvil.
- Todos los textos de la interfaz en español.

## Formato de salida
1. Crea los tres archivos directamente en la carpeta del proyecto.
2. Al terminar, responde con:
  - Un resumen de 3-4 líneas de lo que has creado.
  - Los pasos para probarlo.
  - Cualquier decisión que hayas tomado por tu cuenta y que yo deba revisar.
```

El prompt anterior está escrito en **Markdown**, un formato de texto sencillo para estructurar documentos con títulos, listas, enlaces y bloques de código.

Guía de Markdown: <https://www.markdownguide.org/cheat-sheet/>

Context engineering

Arnés (harness): el sistema que rodea al modelo y le permite actuar como agente (gestiona contexto, herramientas y ejecución de tareas).

Guardarraíles: reglas y controles que limitan sus acciones y validan sus resultados (permisos, aprobaciones y comprobaciones de seguridad).

Contexto: es la información que tiene disponible para realizar una tarea (instrucciones, conversación, archivos, documentación y resultados de herramientas)

La ingeniería de contexto consiste en seleccionar, organizar y actualizar esa información para que el agente reciba lo relevante en cada momento.

AGENTS.md

Es un archivo con instrucciones para orientar a los agentes sobre cómo trabajar en un proyecto. Por defecto, los agentes leerán este fichero de instrucciones. Como recomendación, el contenido del fichero no debe exceder de 30 o 40 líneas, y contener la información principal del proyecto.

¿Qué incluir en **AGENTS.md**?

Stack tecnológico	Convenciones de código	Patrones	Prohibiciones
Estructuras de proyecto	Flujo de trabajo	Testing, CI/CD	Estilo de commits y PRs

<https://agents.md>

Plantilla de fichero **AGENTS.md**

```
None
# AGENTS.md - [Nombre del proyecto]
[Una o dos frases: qué es, para quién y cuál es su objetivo.]

## Stack y estructura
- Tecnologías y versiones clave.
- Qué hay en cada carpeta o archivo importante (solo lo que no es obvio).

## Comandos
- Cómo ejecutar, probar, hacer lint y compilar (comandos exactos, copiables).

## Convenciones
- Estilo de código, nombres, idioma de comentarios y textos.
- Patrones que hay que seguir (y cuál es el archivo de referencia).

## Reglas de dominio / trampas conocidas
- Lo que es fácil hacer mal y el agente no puede deducir leyendo el código.

## Forma de trabajar
- Cuándo planificar antes de tocar código, tamaño de los cambios, qué explicar al terminar.

## Límites
- ✅ Siempre: lo que debe hacer sin preguntar.
- ⚠️ Pregunta antes: dependencias nuevas, archivos nuevos, cambios en el formato de datos...
- ❌ Nunca: lo que no debe tocar bajo ningún concepto.

## Verificación
- Cómo comprobar que un cambio funciona antes de darlo por terminado.
```

El comando `/init` genera el archivo **AGENTS.md**, en base a los ficheros existentes en el proyecto.

```
None
# AGENTS.md - Diario de Estudio

Web estática para registrar sesiones de estudio y motivarse viendo la racha de días seguidos. Proyecto didáctico: el código debe poder entenderlo alguien que empieza a programar.

## Stack y estructura
- HTML, CSS y JavaScript puros: sin frameworks, librerías, npm, bundler ni build.
- `index.html` (estructura), `styles.css` (estilos), `app.js` (lógica y datos).
- Debe funcionar abriendo `index.html` con doble clic (`file:///`): nada de módulos ES (`type="module"`), `fetch` a archivos locales ni nada que requiera servidor.

## Convenciones
- Textos de la interfaz en español.
- Código simple, nombres descriptivos y comentarios solo donde aporten.
- Diseño limpio y responsive; cualquier pantalla nueva debe verse bien en el móvil.

## Datos
- localStorage, clave `diario-estudio-sesiones`: array de `{ date: "AAAA-MM-DD", topic, minutes }`.
- Si cambias la forma de los datos, mantén compatibilidad con lo ya guardado o el usuario perderá sus sesiones.

## Fechas y racha (fácil equivocarse)
- Trabaja siempre con la fecha local del usuario. Nunca uses `toISOString()` ni `new Date("AAAA-MM-DD")`: se interpretan en UTC y desplazan el día.
- Racha = días consecutivos con al menos 1 sesión que terminan hoy. Si hoy no hay sesión pero ayer sí, la racha sigue viva y se cuenta desde ayer.
- Varias sesiones el mismo día cuentan como un solo día. Las fechas futuras no suman.

## Forma de trabajar
- Haz solo lo que se pide: no añadas funcionalidades por tu cuenta.
- Cambios pequeños y enfocados; no reescribas lo que ya funciona.
- Al terminar, resume qué has cambiado y cualquier decisión que deba revisarse.

## Límites
- ✅ Siempre: respetar las reglas de fechas y racha, mantener los textos en español.
- ⚠️ Pregunta antes: crear archivos nuevos, cambiar el formato de los datos guardados.
- ❌ Nunca: añadir dependencias, frameworks o un paso de build.

## Verificación
- No hay tests ni lint. Probar abriendo `index.html` en el navegador.
- Para empezar de cero: DevTools → Application → Local Storage → borrar la clave `diario-estudio-sesiones`.
```

Reglas, Memoria y Documentación

Ficheros adicionales a **AGENTS.md**, con información que explica el proyecto y establece convenciones, límites y criterios que deben seguirse.

Dado que la memoria de un agente es limitada, una buena práctica es crear un fichero de Memoria.

Ejemplo de fichero **MEMORY.md**

```
None

# MEMORY.md - Diario de Estudio
Memoria del proyecto entre sesiones. Máximo ~50 líneas: resume o elimina lo que ya no aporte.

## Estado actual
- v1 funcionando: registrar sesiones (fecha, tema, minutos), racha actual y lista de sesiones.
- Datos en localStorage.

## Decisiones (y por qué)
- Sin backend ni dependencias: cualquiera debe poder abrirlo con doble clic.
- Fecha editable en el formulario: permite registrar días pasados y ver la racha crecer.

## Aprendizajes y errores a evitar
- (vacío por ahora)

## Próximos pasos
- (vacío por ahora)
```

Para que el agente utilice este archivo de memoria, agregamos una nueva sección en nuestro [AGENTS.md](#):

```
None

## Memoria
- Al empezar, lee `MEMORY.md` para conocer el estado del proyecto y las decisiones tomadas.
- Al terminar una tarea, actualízalo: estado actual, decisiones importantes (con su porqué) y errores a evitar.
- Mantenlo breve (máximo ~50 líneas): resume o elimina lo que ya no aporte.
- Si algo se convierte en una regla permanente, propón moverlo a `AGENTS.md` en lugar de dejarlo en la memoria.
- No guardes nunca datos sensibles (claves, tokens, datos personales).
```

Y en la sección de “Límites”, agregamos:

```
None

✅ Siempre: actualizar `MEMORY.md` al terminar cada tarea.
```

Modo Plan (Agente)

El modo plan permite al agente analizar una tarea, consultar el proyecto y proponer los pasos antes de ejecutarlos.

Sirve para aclarar requisitos, detectar riesgos y revisar el enfoque contigo antes de escribir nuevo código o modificar el código existente.

Shift + TAB: alterna entre el modo **Build** y **Plan**.

Cuándo usarlo

- ✓ Tareas grandes
- ✓ Refactors
- ✓ Supervisión de estrategia

Cuándo NO usarlo

- ✗ Tareas pequeñas
- ✗ Correcciones obvias
- ✗ Prototipado rápido

Prompt de ejemplo para utilizar el modo Plan

None

Quiero añadir la "mejor racha": la racha más larga que he conseguido nunca, mostrada junto a la racha actual.

Antes de escribir código, prepárame un plan con:

1. Cómo vas a calcular la mejor racha a partir de las sesiones guardadas, respetando las reglas de fechas y racha de AGENTS.md.
2. Qué archivos vas a modificar y qué cambia en cada uno.
3. Los casos límite y las dudas que debo decidir yo antes de empezar.
4. Qué actualizarías en AGENTS.md y en MEMORY.md.

Súmate a nuestro directo del día 2

[IR AL DIRECTO](#)